plugin 列表：接口文档

> 各个插件的配置属性

auth_basic

> 认证插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/auth_basic
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/auth_basic' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

```
{
    "title": "work with route or service object",
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "hide_credentials": {
            "type": "boolean",
            "default": false
        },
        "_meta": {
            "type": "object",
            "properties": {
                "priority": {
                    "description": "priority of plugins by customized order",
                    "type": "integer"
                },
                "filter": {
                    "description": "filter determines whether the plugin needs to be executed at runtime",
```

```
                "items": {
                    "type": "array"
                },
                "type": "array",
                "maxItems": 20
            },
            "error_page": {
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
    },
    "disable": {
        "type": "boolean"
    }
  }
}
```

插件属性解释

名称	解释	类型和示例
hide_credentials	1．设置为 false 时，将透传给 Upstream。	bool，默认值，false

	2. 设置为 true 时，将清除掉对应的信息，不传递给Upstream。 3. 说明：如果header，query 都配置了，按优先级 header > query，取优先级较高的	
username	1. 为 Consumer 配置的用户名，用于识别用户身份，这个 username 应该是不相同的。 2. 如果多个 Consumer 使用了相同的 username，将会出现请求匹配异常	string
password	用户密码	string

配置插件示例

> 新增 consumer, 配置为使用 auth_basic 插件

1. 配置使用 auth_basic 插件并设置用户名密码

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/consumers' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "username": "CAtest_auth_basic",
    "plugins": {
        "auth_basic": {
            "password": "auth_basic",
            "username": "auth_basic"
        }
    }
}'
```

2. 设置成功

```
{
    "node": {
        "value": {
            "plugins": {
                "auth_basic": {
                    "username": "auth_basic",
                    "password": "auth_basic"
                }
            },
            "update_time": 1676255420,
            "username": "CAtest_auth_basic",
            "create_time": 1676255420
        },
        "key": "/search/consumers/CAtest_auth_basic"
    },
    "header": {},
    "action": "set"
}
```

auth_key

> 认证插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/auth_key
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/auth_key' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "disable": {
            "type": "boolean"
        },
        "_meta": {
            "type": "object",
            "properties": {
                "priority": {
                    "description": "priority of plugins by customized order",
                    "type": "integer"
                },
                "filter": {
                    "description": "filter determines whether the plugin needs to be executed at runtime",
                    "items": {
                        "type": "array"
                    },
                    "type": "array",
                    "maxItems": 20
                },
                "error_page": {
```

```
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
    },
    "query": {
        "type": "string",
        "default": "apikey"
    },
    "hide_credentials": {
        "type": "boolean",
        "default": false
    },
    "header": {
        "type": "string",
        "default": "apikey"
    }
  }
}
```

插件属性解释

名称	解释	类型	示例

header	设置 apikey 从 header 中指定字段获取	string，默认值: apikey	例如：该值指定为 xapikey 时，apikey 从Header [”xapikey”] 中获取
query	设置 apikey 从请求参数中获取 key	string，默认值: apikey	例如：该值指定为 xapikey 时，apikey 从 url? xapikey=xxx 的args [”xapikey”] 获取
hide_credentials	1. 设置为 false 时，将透传给 Upstream。 2. 设置为 true 时，将清除掉对应的信息，不传递给Upstream。 3. 说明：如果header，query 都配置了，按优先级 header > query，取优先级较高的	bool，默认值: false	

配置插件示例

1. 新增路由 /hello , 为该路由添加 auth_key 插件

```
curl –L –X PUT ’http://192.168.100.73:9990/edge/admin/consumers’ –H ’X–API–KEY:
f9357106bff442f89d4de7169c37c61e’ –H ’Content–Type: application/json’ ––data ’{
    ”username”: ”CAtest_auth_basic”,
    ”plugins”: {
        ”auth_basic”: {
            ”password”: ”auth_basic”,
            ”username”: ”auth_basic”
        }
    }
}’
```

2. 添加路由成功响应

```
{
    ”node”: {
        ”value”: {
            ”plugins”: {
                ”auth_key”: {
                    ”hide_credentials”: false,
                    ”query”: ”apikey”,
                    ”header”: ”apikey”
                }
            },
            ”id”: ”3001”,
            ”update_time”: 1676269629,
            ”create_time”: 1675905156,
            ”upstream”: {
                ”pass_host”: ”pass”,
                ”scheme”: ”http”,
                ”nodes”: {
                    ”127.0.0.1:8111”: 1
                },
                ”type”: ”roundrobin”,
```

```
                "hash_on": "vars"
            },
            "status": 1,
            "uri": "/hello",
            "priority": 0
        },
        "key": "/search/routes/3001"
    },
    "header": {},
    "action": "set"
}
```


3. 新增 consumer, 设置 auth_key 插件的 apikey 为 xxxxyyyyzzzz


```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/consumers' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "username": "CAtest_auth_key",
    "plugins": {
        "auth_key": {
            "key": "xxxxyyyyzzzz"
        }
    }
}'
```


4. 设置 apikey 成功的响应


```
{
    "node": {
        "value": {
            "plugins": {
                "auth_key": {
                    "key": "xxxxyyyyzzzz"
                }
            },
            "update_time": 1676269856,
            "username": "CAtest_auth_key",
            "create_time": 1676269856
        },
        "key": "/search/consumers/CAtest_auth_key"
    },
    "header": {},
    "action": "set"
}
```


5. 测试新路由 /hello, 使用新 apikey


```
curl -L -X GET 'http://192.168.100.73:9980/hello' -H 'apikey: xxxxyyyyzzzz'
```


6. 调用成功


```
192.168.100.73:8112/hello traceid=:
```


security_common_args

> 安全参数插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/security_common_args
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/security_common_args' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "message": {
            "default": "Your request args is not allowed",
            "minLength": 1,
            "type": "string",
            "maxLength": 1024
        },
        "allowlist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "denylist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "status": {
```

```
        "default": 403,
        "minimum": 200,
        "type": "integer",
        "maximum": 599
    },
    "_meta": {
        "type": "object",
        "properties": {
            "priority": {
                "description": "priority of plugins by customized order",
                "type": "integer"
            },
            "filter": {
                "description": "filter determines whether the plugin needs to be executed at runtime",
                "items": {
                    "type": "array"
                },
                "type": "array",
                "maxItems": 20
            },
            "error_page": {
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
    },
    "disable": {
        "type": "boolean"
    }
```

```
    }
}
```

插件属性解释

名称	解释	类型	示例
allowlist	白名单列表，匹配请求参数中的关键字，命中后结束当前插件	array[string]	注意： 如果allowlist，denylist都配置了，按优先级 allowlist > denylist，依次检查
denylist	黑名单列表，匹配请求参数中的关键字，命中后拦截用户请求	array[string]	
status	拦截后，响应的状态码	integer，默认值: 403	
message	拦截后，响应的信息	string，默认值，”Your request args is not allowed”	

配置插件示例

1．添加全局插件, 验证所有的参数

```
curl –L –X PUT ’http://192.168.100.73:9990/edge/admin/global_rules/6001’ –H ’X–API–KEY:
f9357106bff442f89d4de7169c37c61e’ –H ’Content–Type: application/json’ ––data ’{
    ”desc”: ”security_common”,
    ”plugins”: {
        ”security_common_args”: {
            ”denylist”: [
                ”jndi\\:(?:ldap|rmi|iiop|iiopname|corbaname|dns|nis)”,
                ”\\.\\./”,
                ”\\:\\$”,
                ”\\$\\{”,
                ”\\Wselect\\W.+\\W(?:from|limit)\\W”,
                ”(?:(union(.*?)select))”,
                ”sleep\\((\\s*)(\\d*)(\\s*)\\)”,
                ”benchmark\\((.*)\\,(.*)\\)”,
                ”base64_decode\\(”,
                ”(?:from\\W+information_schema\\W)”,
                ”(?:(?:current_)user|database|schema|connection_id)\\s*\\(”,
                ”(?:etc\\/\\W*passwd)”,
                ”into(\\s+)+(?:dump|out)file\\s*”,
                ”group\\s+by.+\\(”,
                ”xwork.MethodAccessor”,
                ”(?:
define|eval|file_get_contents|include|require|require_once|shell_exec|phpinfo|system|passthru|preg_\\w+|execute|echo|print|p
```

2. 新增成功响应


```
{
    ”node”: {
        ”value”: {
            ”plugins”: {
                ”security_common_args”: {
                    ”message”: ”Your request args is not allowed”,
                    ”status”: 403,
                    ”denylist”: [
                        ”jndi\\:(?:ldap|rmi|iiop|iiopname|corbaname|dns|nis)”,
                        ”\\.\\./”,
                        ”\\:\\$”,
                        ”\\$\\{”,
                        ”\\Wselect\\W.+\\W(?:from|limit)\\W”,
                        ”(?:(union(.*?)select))”,
                        ”sleep\\((\\s*)(\\d*)(\\s*)\\)”,
                        ”benchmark\\((.*)\\,(.*)\\)”,
                        ”base64_decode\\(”,
                        ”(?:from\\W+information_schema\\W)”,
                        ”(?:(?:current_)user|database|schema|connection_id)\\s*\\(”,
                        ”(?:etc\\/\\W*passwd)”,
                        ”into(\\s+)+(?:dump|out)file\\s*”,
                        ”group\\s+by.+\\(”,
                        ”xwork.MethodAccessor”,
                        ”(?:
define|eval|file_get_contents|include|require|require_once|shell_exec|phpinfo|system|passthru|preg_\\w+|execute|echo|print|p
```


3. 测试路由 /hello, 使用新被禁止的字符 $ 和 {

```
curl –L –X GET ’http://192.168.100.73:9980/hello?test=$\{’
```

4. 验证已经成功禁止非法字符

```
{
    ”message”: ”Your request args is not allowed”
}
```

security_common_body

> 安全 body 插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/security_common_body
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/security_common_body' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应


```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "message": {
            "default": "Your request data is not allowed",
            "minLength": 1,
            "type": "string",
            "maxLength": 1024
        },
        "bypass_hugebody": {
            "type": "boolean",
            "default": false
        },
        "allowlist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "denylist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "maxsize": {
            "minimum": 1,
```

```
        "type": "integer",
        "default": 4096
    },
    "status": {
        "default": 403,
        "minimum": 200,
        "type": "integer",
        "maximum": 599
    },
    "_meta": {
        "type": "object",
        "properties": {
            "priority": {
                "description": "priority of plugins by customized order",
                "type": "integer"
            },
            "filter": {
                "description": "filter determines whether the plugin needs to be executed at runtime",
                "items": {
                    "type": "array"
                },
                "type": "array",
                "maxItems": 20
            },
            "error_page": {
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
```

```
        },
        "disable": {
            "type": "boolean"
        }
    }
}
```

插件属性解释

名称	解释	类型	示例
allowlist	白名单列表，匹配body中的关键字，命中后结束当前插件	array[string]	注意： 如果allowlist，denylist都配置了，按优先级 allowlist > denylist，依次检查 如果请求体较大时应避免使用这个规则
denylist	黑名单列表，匹配body中的关键字，命中后拦截用户请求	array[string]	
maxsize	请求体大小，超过限制大小的不进行匹配	integer，默认值: 4096	
status	拦截后，响应的状态码	integer，默认值: 403	
message	拦截后，响应的信息	string，默认值: "Your request body is not allowed"	
bypass_hugebody	当设置为 true 时，如果请求体大小超过maxsize 时，将绕过检查。	boolean，默认值: false	

配置插件示例

1. 添加全局插件, 验证所有的 body

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/global_rules/6002' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "desc": "security_common",
    "plugins": {
        "security_common_body": {
            "bypass_hugebody": true,
            "denylist": [
```

```
                "jndi\\:(?:ldap|rmi|iiop|iiopname|corbaname|dns|nis)",
                "\\.\\./",
                "\\:\\$",
                        "\\$\\{",
                "(?:^|\\W)select\\W.+\\W(?:from|limit)\\W",
                "(?:(union(.*?)select))",
                "sleep\\((\\s*)(\\d*)(\\s*)\\)",
                "benchmark\\((.*)\\,(.*)\\)",
                "base64_decode\\(",
                "(?:from\\W+information_schema\\W)",
                "(?:(?:current_)user|database|schema|connection_id)\\s*\\(",
                "(?:etc\\/\\W*passwd)",
                "into(\\s+)+(?:dump|out)file\\s*",
                "group\\s+by.+\\(",
                "xwork.MethodAccessor",
                "(?:
define|eval|file_get_contents|include|require|require_once|shell_exec|phpinfo|system|passthru|preg_\\w+|execute|echo|print|p
```

2. 新增成功响应

```
{
    "node": {
        "value": {
            "plugins": {
                "security_common_body": {
                    "bypass_hugebody": true,
                    "denylist": [
                        "jndi\\:(?:ldap|rmi|iiop|iiopname|corbaname|dns|nis)",
                        "\\.\\./",
                        "\\:\\$",
                        "\\$\\{",
                        "(?:^|\\W)select\\W.+\\W(?:from|limit)\\W",
                        "(?:(union(.*?)select))",
                        "sleep\\((\\s*)(\\d*)(\\s*)\\)",
                        "benchmark\\((.*)\\,(.*)\\)",
                        "base64_decode\\(",
                        "(?:from\\W+information_schema\\W)",
                        "(?:(?:current_)user|database|schema|connection_id)\\s*\\(",
                        "(?:etc\\/\\W*passwd)",
                        "into(\\s+)+(?:dump|out)file\\s*",
                        "group\\s+by.+\\(",
                        "xwork.MethodAccessor",
                        "(?:
define|eval|file_get_contents|include|require|require_once|shell_exec|phpinfo|system|passthru|preg_\\w+|execute|echo|print|p
```

3. 测试路由 /hello, 使用新被禁止的字符 $ 和 {

```
curl -L -X POST 'http://192.168.100.73:9980/hello' --data 'xxx$_GET[xxx'
```

4. 验证已经成功禁止非法字符

```
{
    "message": "Your request data is not allowed"
}
```

security_common_cookie

> 安全 cookie 插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/security_common_cookie
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/security_common_cookie' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应


```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "message": {
            "default": "Your request cookie is not allowed",
            "minLength": 1,
            "type": "string",
            "maxLength": 1024
        },
        "allowlist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "denylist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
```

```
        "minItems": 1
    },
    "_meta": {
        "type": "object",
        "properties": {
            "priority": {
                "description": "priority of plugins by customized order",
                "type": "integer"
            },
            "filter": {
                "description": "filter determines whether the plugin needs to be executed at runtime",
                "items": {
                    "type": "array"
                },
                "type": "array",
                "maxItems": 20
            },
            "error_page": {
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
    },
    "status": {
        "default": 403,
        "minimum": 200,
        "type": "integer",
        "maximum": 599
    },
```

```
        "disable": {
            "type": "boolean"
        },
        "bypass_missing": {
            "type": "boolean",
            "default": false
        }
    }
}
```

插件属性解释

名称	解释	类型	示例
allowlist	白名单列表，匹配 cookie 中的关键字，命中后结束当前插件	array[string]	注意： 如果allowlist，denylist都配置了，按优先级 allowlist > denylist，依次检查
denylist	黑名单列表，匹配 cookie 中的关键字，命中后拦截用户请求	array[string]	
status	拦截后，响应的状态码	integer，默认值: 403	
message	拦截后，响应的信息	string，默认值: "Your request cookie is not allowed"	
bypass_missing	当设置为 true 时，如果 cookie 请求头不存在或格式有误时，将绕过检查。	boolean，默认值: false	

配置插件示例

1. 添加全局插件, 验证所有的 cookie

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/global_rules/6003' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "desc": "security_common",
    "plugins": {
        "security_common_cookie": {
            "bypass_missing": true,
            "denylist": [
                "jndi\\:(?:ldap|rmi|iiop|iiopname|corbaname|dns|nis)",
                "\\.\\./",
                "\\:\\$",
```

```
                "\\$\\{",
                "(?:^|\\W)select\\W.+\\W(?:from|limit)\\W",
                "(?:(union(.*?)select))",
                "sleep\\((\\s*)(\\d*)(\\s*)\\)",
                "benchmark\\((.*)\\,(.*)\\)",
                "base64_decode\\(",
                "(?:from\\W+information_schema\\W)",
                "(?:(?:current_)user|database|schema|connection_id)\\s*\\(",
                "(?:etc/\\W*passwd)",
                "into(\\s+)+(?:dump|out)file\\s*",
                "group\\s+by.+\\(",
                "xwork.MethodAccessor",
                "(?:
define|eval|file_get_contents|include|require|require_once|shell_exec|phpinfo|system|passthru|preg_\\w+|execute|echo|print|p
```

2. 新增成功响应

```
{
    "node": {
        "value": {
            "plugins": {
                "security_common_cookie": {
                    "bypass_missing": true,
                    "denylist": [
                        "jndi\\:(?:ldap|rmi|iiop|iiopname|corbaname|dns|nis)",
                        "\\.\\./",
                        "\\:\\$",
                        "\\$\\{",
                        "(?:^|\\W)select\\W.+\\W(?:from|limit)\\W",
                        "(?:(union(.*?)select))",
                        "sleep\\((\\s*)(\\d*)(\\s*)\\)",
                        "benchmark\\((.*)\\,(.*)\\)",
                        "base64_decode\\(",
                        "(?:from\\W+information_schema\\W)",
                        "(?:(?:current_)user|database|schema|connection_id)\\s*\\(",
                        "(?:etc/\\W*passwd)",
                        "into(\\s+)+(?:dump|out)file\\s*",
                        "group\\s+by.+\\(",
                        "xwork.MethodAccessor",
                        "(?:
define|eval|file_get_contents|include|require|require_once|shell_exec|phpinfo|system|passthru|preg_\\w+|execute|echo|print|p
```

3. 测试路由 /hello, 使用新被禁止的字符 $ 和 {

```
curl -L -X GET 'http://192.168.100.73:9980/hello?test' -H 'Cookie: DEVICEID=f301M6ud0R base64_decode
(QBxnBCN0MV-RO9Vel9vgcAcBaLq1y'
```

4. 验证已经成功禁止非法字符

```
{
    "message": "Your request cookie is not allowed"
}
```

security_common_referer

> 安全 referer 插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/security_common_referer
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/security_common_referer' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应


```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "disable": {
            "type": "boolean"
        },
        "_meta": {
            "type": "object",
            "properties": {
                "priority": {
                    "description": "priority of plugins by customized order",
                    "type": "integer"
                },
                "filter": {
                    "description": "filter determines whether the plugin needs to be executed at runtime",
                    "items": {
                        "type": "array"
                    },
                    "type": "array",
                    "maxItems": 20
                },
                "error_page": {
                    "type": "object",
                    "properties": {
                        "headers": {
```

```
                    "type": "object"
                },
                "status": {
                    "minimum": 0,
                    "type": "integer",
                    "maximum": 599
                },
                "body": {
                    "oneOf": [
                        {
                            "type": "string"
                        },
                        {
                            "type": "object"
                        }
                    ]
                }
            }
        },
        "error_response": {
            "oneOf": [
                {
                    "type": "string"
                },
                {
                    "type": "object"
                }
            ]
        }
    }
},
"bypass_missing": {
    "type": "boolean",
    "default": false
},
"whitelist": {
    "minItems": 1,
    "type": "array",
    "items": {
        "type": "string",
        "pattern": "^\\*?[0-9a-zA-Z-._\\[\\]:]+$"
    }
},
"allowlist": {
    "items": {
        "type": "string",
        "minLength": 1
    },
    "type": "array",
    "minItems": 1
},
"denylist": {
    "items": {
        "type": "string",
        "minLength": 1
    },
    "type": "array",
    "minItems": 1
```

```
        },
        "status": {
            "default": 403,
            "minimum": 200,
            "type": "integer",
            "maximum": 599
        },
        "blacklist": {
            "minItems": 1,
            "type": "array",
            "items": {
                "type": "string",
                "pattern": "^\\*?[0-9a-zA-Z-._\\[\\]:]+$"
            }
        },
        "message": {
            "default": "Your request referer host is not allowed",
            "minLength": 1,
            "type": "string",
            "maxLength": 1024
        }
    }
}
```

插件属性解释

名称	解释	类型	示例
allowlist	白名单列表，匹配 Referer 中的关键字，命中后结束当前插件	array[string]	注意： 如果配置了多个列表，按优先级 allowlist > denylist > whitelist > blacklist，依次检查
denylist	黑名单列表，匹配 Referer 中的关键字，命中后拦截用户请求	array[string]	
whitelist	host 的白名单列表，匹配 Referer 中的 host关键字，命中后结束当前插件	array[string]	
blacklist	host 的黑名单列表，匹配 Referer 中的 host关键字，命中后拦截用户请求。	array[string]	
status	拦截后，响应的状态码	integer，默认值: 403	
message	拦截后，响应的信息	string，默认值: "Your request referer is not allowed"	

bypass_missing	当设置为 true 时，如果 Referer 请求头不存在或格式有误时，将绕过检查。	boolean，默认值: false	

配置插件示例

1. 添加全局插件, 验证所有的 referer

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/global_rules/6004' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "desc": "security_common",
    "plugins": {
        "security_common_uri": {
            "denylist": [
                "\\.(?:svn|htaccess|bash_history)"
            ]
        }
    }
}'
```

2. 新增成功响应

```
{
    "node": {
        "value": {
            "plugins": {
                "security_common_uri": {
                    "message": "Your request uri is not allowed",
                    "status": 403,
                    "denylist": [
                        "\\.(?:svn|htaccess|bash_history)"
                    ]
                }
            },
            "id": "6004",
            "create_time": 1676275545,
            "desc": "security_common",
            "update_time": 1676278047
        },
        "key": "/search/global_rules/6004"
    },
    "header": {},
    "action": "set"
}
```

3. 测试路由 /hello, 使用新被禁止的字符 svn

```
curl -L -X GET 'http://192.168.100.73:9980/hello?test' -H 'Referer: test.svn'
```

4. 验证已经成功禁止非法字符

```
{
    "message": "Your request referer host is not allowed"
}
```

security_common_uri

> 安全 uri 插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/security_common_uri
- **Method**：`GET`
- **需要登录**：`是`
- **需要鉴权**：`是`
- **输入数据**：`通过接口方式传输`
- **输出数据**：`jsonschema格式格式返回`
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/security_common_uri' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应

```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "message": {
            "default": "Your request uri is not allowed",
            "minLength": 1,
            "type": "string",
            "maxLength": 1024
        },
        "allowlist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "denylist": {
            "items": {
```

```
            "type": "string",
            "minLength": 1
        },
        "type": "array",
        "minItems": 1
    },
    "status": {
        "default": 403,
        "minimum": 200,
        "type": "integer",
        "maximum": 599
    },
    "_meta": {
        "type": "object",
        "properties": {
            "priority": {
                "description": "priority of plugins by customized order",
                "type": "integer"
            },
            "filter": {
                "description": "filter determines whether the plugin needs to be executed at runtime",
                "items": {
                    "type": "array"
                },
                "type": "array",
                "maxItems": 20
            },
            "error_page": {
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
```

```
                ]
            }
        }
    },
    "disable": {
        "type": "boolean"
    }
  }
}
```

插件属性解释

名称	解释	类型	示例
allowlist	白名单列表，匹配 uri 中的关键字，命中后结束当前插件	array[string]	注意： 如果 allowlist，denylist 都配置了，按优先级 allowlist > denylist，依次检查
denylist	黑名单列表，匹配 uri 中的关键字，命中后拦截用户请求	array[string]	
status	拦截后，响应的状态码	integer，默认值: 403	
message	拦截后，响应的信息	string，默认值: "Your request uri is not allowed"	
bypass_missing	当设置为 true 时，如果 Referer 请求头不存在或格式有误时，将绕过检查。	boolean，默认值: false	

配置插件示例

1. 添加全局插件, 验证所有的 uri

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/global_rules/6005' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "desc": "security_common",
    "plugins": {
        "security_common_uri": {
            "denylist": [
                "\\.(?:svn|htaccess|bash_history)"
            ]
```

```
        }
    }
}'
```

2. 新增成功响应

```
{
    "node": {
        "value": {
            "plugins": {
                "security_common_uri": {
                    "message": "Your request uri is not allowed",
                    "status": 403,
                    "denylist": [
                        "\\.(?:svn|htaccess|bash_history)"
                    ]
                }
            },
            "id": "6005",
            "create_time": 1676276656,
            "desc": "security_common",
            "update_time": 1676276656
        },
        "key": "/search/global_rules/6005"
    },
    "header": {},
    "action": "set"
}
```

3. 测试路由 /hello, 使用新被禁止的字符 .svn

```
curl -L -X GET 'http://192.168.100.73:9980/hello1?x.svn'
```

4. 验证已经成功禁止非法字符

```
{
    "message": "Your request uri is not allowed"
}
```

security_common_useragent

> 安全 uri 插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/security_common_useragent
- **Method**：GET
- **需要登录**：是
- **需要鉴权**：是
- **输入数据**：通过接口方式传输
- **输出数据**：jsonschema格式格式返回
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/security_common_useragent' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应


```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "message": {
            "default": "Your request useragent is not allowed",
            "minLength": 1,
            "type": "string",
            "maxLength": 1024
        },
        "allowlist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "denylist": {
            "items": {
                "type": "string",
                "minLength": 1
            },
            "type": "array",
            "minItems": 1
        },
        "_meta": {
            "type": "object",
            "properties": {
                "priority": {
                    "description": "priority of plugins by customized order",
                    "type": "integer"
```

```
            },
            "filter": {
                "description": "filter determines whether the plugin needs to be executed at runtime",
                "items": {
                    "type": "array"
                },
                "type": "array",
                "maxItems": 20
            },
            "error_page": {
                "type": "object",
                "properties": {
                    "headers": {
                        "type": "object"
                    },
                    "status": {
                        "minimum": 0,
                        "type": "integer",
                        "maximum": 599
                    },
                    "body": {
                        "oneOf": [
                            {
                                "type": "string"
                            },
                            {
                                "type": "object"
                            }
                        ]
                    }
                }
            },
            "error_response": {
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
    },
    "status": {
        "default": 403,
        "minimum": 200,
        "type": "integer",
        "maximum": 599
    },
    "disable": {
        "type": "boolean"
    },
    "bypass_missing": {
        "type": "boolean",
        "default": false
    }
}
```

```
}
```

插件属性解释

名称	解释	类型	示例
allowlist	白名单列表，匹配 User-Agent 中的关键字，命中后结束当前插件	array[string]	注意： 如果 allowlist，denylist 都配置了，按优先级 allowlist > denylist，依次检查
denylist	黑名单列表，匹配 User-Agent 中的关键字，命中后拦截用户请求	array[string]	
status	拦截后，响应的状态码	integer，默认值: 403	
message	拦截后，响应的信息	string，默认值: "Your request useragent is not allowed"	
bypass_missing	当设置为 true 时，如果 User-Agent 请求头不存在或格式有误时，将绕过检查。	boolean，默认值: false	

配置插件示例

1. 添加全局插件, 验证所有的 uri

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/global_rules/6006' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' -H 'Content-Type: application/json' --data '{
    "desc": "security_common",
    "plugins": {
        "security_common_useragent": {
            "bypass_missing": true,
            "denylist": [
                "\\.(?:svn|htaccess|bash_history)"
            ]
        }
    }
}'
```

2. 新增成功响应

```
{
    "node": {
        "value": {
```

```
            "plugins": {
                "security_common_useragent": {
                    "message": "Your request useragent is not allowed",
                    "denylist": [
                        "\\.(?:svn|htaccess|bash_history)"
                    ],
                    "status": 403,
                    "bypass_missing": true
                }
            },
            "id": "6006",
            "create_time": 1676277372,
            "desc": "security_common",
            "update_time": 1676277372
        },
        "key": "/search/global_rules/6006"
    },
    "header": {},
    "action": "set"
}
```


3. 测试路由 /hello, 使用新被禁止的字符 .svn

```
curl -L -X GET 'http://192.168.100.73:9980/hello?test' -H 'User-Agent: PostmanRuntime/7.30.0 test.svn'
```

4. 验证已经成功禁止非法字符


```
{
    "message": "Your request useragent is not allowed"
}
```


monitor

> 安全 uri 插件

获取插件的配置属性

允许已授权的用户通过此接口根据插件名称获取该插件的的配置属性

- **URL**：/edge/admin/plugins/monitor
- **Method**：`GET`
- **需要登录**：`是`
- **需要鉴权**：`是`
- **输入数据**：`通过接口方式传输`
- **输出数据**：`jsonschema格式格式返回`
- **接口调用出于安全考虑带密码认证，需要在请求头中添加 X-API-KEY 属性**

请求示例

```
curl -L -X GET 'http://192.168.100.73:9990/edge/admin/plugins/monitor' -H 'X-API-KEY: f9357106bff442f89d4de7169c37c61e'
```

成功响应


```
{
    "$comment": "injected plugin schema",
    "type": "object",
    "properties": {
        "prefer_name": {
            "type": "boolean",
            "default": false
        },
        "_meta": {
            "type": "object",
            "properties": {
                "priority": {
                    "description": "priority of plugins by customized order",
                    "type": "integer"
                },
                "filter": {
                    "description": "filter determines whether the plugin needs to be executed at runtime",
                    "items": {
                        "type": "array"
                    },
                    "type": "array",
                    "maxItems": 20
                },
                "error_page": {
                    "type": "object",
                    "properties": {
                        "headers": {
                            "type": "object"
                        },
                        "status": {
                            "minimum": 0,
                            "type": "integer",
                            "maximum": 599
                        },
                        "body": {
                            "oneOf": [
                                {
                                    "type": "string"
                                },
                                {
                                    "type": "object"
                                }
                            ]
                        }
                    }
                },
                "error_response": {
```

```
                "oneOf": [
                    {
                        "type": "string"
                    },
                    {
                        "type": "object"
                    }
                ]
            }
        }
    },
    "disable": {
        "type": "boolean"
    }
  }
}
```

插件属性解释

名称	解释	类型	示例
prefer_name	当设置为true时，将使用路由或服务的 name 来标识所命中的路由或服务，否则使用 id	boolean，默认值: false	说明： 设置 prefer_name 为 true 时，路由或服务的名称没有唯一性，所以需要规范路由和服务的命名，以便于区分。

监控指标

名称	解释	示例
edge_bandwidth	带宽	type="egress" 出带宽 type="ingress" 入带宽
edge_http_latency_bucket	耗时计数	统计1,2,5,10,20,50,100,200,500,1000,2000,5000,10000,30000,60000ms以内的请求耗时数
edge_http_latency_count	各耗时来	

	源计数	
edge_http_latency_sum	各耗时来源总耗时	type=”edge” 耗时来源edge type=”upstream” 耗时来源upstream type=”request” 耗时来源request，约为 edge 与 upstream 之和
edge_http_requests_total	总请求数	
edge_http_status	各状态信息统计	
edge_nginx_http_current_connections	nginx连接信息	
edge_nginx_metric_errors_total	错误数	
edge_node_info	结点信息	
edge_shared_dict_capacity_bytes	各内存分配空间大小	
edge_shared_dict_free_space_bytes	各内存剩余空间大小	

配置插件示例

1．添加插件，新增路由的相关统计信息

```
curl -L -X PUT 'http://192.168.100.73:9990/edge/admin/routes/3001' -H 'X-API-KEY:
f9357106bff442f89d4de7169c37c61e' --data '{
    "uri": "/helloworld",
    "plugins": {
        "monitor": {}
    },
    "upstream": {
        "nodes": {
            "127.0.0.1:8111": 1
        },
```

```
        "type": "roundrobin"
    }
}'
```

2. 新增成功响应

```
{
    "node": {
        "value": {
            "plugins": {
                "monitor": {
                    "prefer_name": false
                }
            },
            "id": "3001",
            "update_time": 1676279029,
            "create_time": 1675905156,
            "upstream": {
                "pass_host": "pass",
                "scheme": "http",
                "nodes": {
                    "127.0.0.1:8111": 1
                },
                "type": "roundrobin",
                "hash_on": "vars"
            },
            "status": 1,
            "uri": "/helloworld",
            "priority": 0
        },
        "key": "/search/routes/3001"
    },
    "header": {},
    "action": "set"
}
```

3. 测试路由 /helloworld, 多请求几次

```
curl -L -X GET 'http://192.168.100.73:9980/helloworld'
```

4. 查看监控信息

```
curl -L -X GET 'http://192.168.100.73:9990/edge/monitor/metrics'
```

```
# HELP edge_bandwidth Total bandwidth in bytes consumed per service in EDGE
# TYPE edge_bandwidth counter
edge_bandwidth{type="egress",route="3001",service="",consumer="",node=""} 651
edge_bandwidth{type="ingress",route="3001",service="",consumer="",node=""} 279
# HELP edge_http_latency HTTP request latency in milliseconds per service in EDGE
# TYPE edge_http_latency histogram
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="1"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="2"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="5"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="10"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="20"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="50"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="100"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="200"} 3
```

```
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="500"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="1000"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="2000"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="5000"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="10000"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="30000"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="60000"} 3
edge_http_latency_bucket{type="edge",route="3001",service="",consumer="",node="",le="+Inf"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="1"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="2"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="5"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="10"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="20"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="50"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="100"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="200"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="500"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="1000"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="2000"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="5000"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="10000"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="30000"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="60000"} 3
edge_http_latency_bucket{type="request",route="3001",service="",consumer="",node="",le="+Inf"} 3
edge_http_latency_count{type="edge",route="3001",service="",consumer="",node=""} 3
edge_http_latency_count{type="request",route="3001",service="",consumer="",node=""} 3
edge_http_latency_sum{type="edge",route="3001",service="",consumer="",node=""} 0
edge_http_latency_sum{type="request",route="3001",service="",consumer="",node=""} 0
# HELP edge_http_requests_total The total number of client requests since EDGE started
# TYPE edge_http_requests_total gauge
edge_http_requests_total 97
# HELP edge_http_status HTTP status codes per service in EDGE
# TYPE edge_http_status counter
edge_http_status{code="401",route="3001",matched_uri="/helloworld",matched_host="",service="",consumer="",
node=""} 3
# HELP edge_nginx_http_current_connections Number of HTTP connections
# TYPE edge_nginx_http_current_connections gauge
edge_nginx_http_current_connections{state="accepted"} 91
edge_nginx_http_current_connections{state="active"} 1
edge_nginx_http_current_connections{state="handled"} 91
edge_nginx_http_current_connections{state="reading"} 0
edge_nginx_http_current_connections{state="waiting"} 0
edge_nginx_http_current_connections{state="writing"} 1
# HELP edge_nginx_metric_errors_total Number of nginx-lua-prometheus errors
# TYPE edge_nginx_metric_errors_total counter
edge_nginx_metric_errors_total 0
# HELP edge_node_info Info of EDGE node
# TYPE edge_node_info gauge
edge_node_info{hostname="jdjk5.novalocal"} 1
# HELP edge_shared_dict_capacity_bytes The capacity of each nginx shared DICT since EDGE start
# TYPE edge_shared_dict_capacity_bytes gauge
edge_shared_dict_capacity_bytes{name="balancer_ewma"} 10485760
edge_shared_dict_capacity_bytes{name="balancer_ewma_last_touched_at"} 10485760
edge_shared_dict_capacity_bytes{name="balancer_ewma_locks"} 10485760
edge_shared_dict_capacity_bytes{name="fdict_cache"} 104857600
edge_shared_dict_capacity_bytes{name="fdict_cache_locks"} 1048576
edge_shared_dict_capacity_bytes{name="fdict_cache_version"} 1048576
edge_shared_dict_capacity_bytes{name="internal_status"} 10485760
```

```
edge_shared_dict_capacity_bytes{name="lrucache_lock"} 10485760
edge_shared_dict_capacity_bytes{name="memory_cache"} 52428800
edge_shared_dict_capacity_bytes{name="monitor_metrics"} 10485760
edge_shared_dict_capacity_bytes{name="upstream_healthcheck"} 10485760
edge_shared_dict_capacity_bytes{name="worker_events"} 10485760
# HELP edge_shared_dict_free_space_bytes The free space of each nginx shared DICT since EDGE start
# TYPE edge_shared_dict_free_space_bytes gauge
edge_shared_dict_free_space_bytes{name="balancer_ewma"} 10412032
edge_shared_dict_free_space_bytes{name="balancer_ewma_last_touched_at"} 10412032
edge_shared_dict_free_space_bytes{name="balancer_ewma_locks"} 10412032
edge_shared_dict_free_space_bytes{name="fdict_cache"} 104222720
edge_shared_dict_free_space_bytes{name="fdict_cache_locks"} 1032192
edge_shared_dict_free_space_bytes{name="fdict_cache_version"} 1032192
edge_shared_dict_free_space_bytes{name="internal_status"} 10412032
edge_shared_dict_free_space_bytes{name="lrucache_lock"} 10412032
edge_shared_dict_free_space_bytes{name="memory_cache"} 52113408
edge_shared_dict_free_space_bytes{name="monitor_metrics"} 10383360
edge_shared_dict_free_space_bytes{name="upstream_healthcheck"} 10412032
edge_shared_dict_free_space_bytes{name="worker_events"} 10407936
```